Project Proposal – Football tactics to match simulation

Jakob Persson 4DT903

Jp223uy@student.lnu.se

Objective

- *Which domain and the problem you will address?*

The objective for this project is to develop a software tool that transforms football tactic models into Python simulations using Model-Driven Engineering. The problem is that formations, tactical instructions and player attributes must be translated into simulation before the potential effects can be evaluated.

There are three domains:

- Football tactics: Teams, formations, players, attributes, defensive and offensive instructions.
- Simulation: Rules for simulating possession, passing situations, player movement and scoring opportunities.
- Match results: Statistics produced by the simulation, such as possession, pass success, chances and goals.

For example, a user could model a 4-2-1-3 formation with counterattack and low press. The software would transform this model into Python simulation that estimates possession, passing success and scoring chances. The simulation will use simplified probability rules rather than reproducing an entire football match.

Models

- *What models and metamodels are needed?*

The project would probably require at least these three metamodels:

1. **Football Tactics Metamodel (FootballMM)**
   FootballMM represents the football domain and defines the concepts used to describe a team, this involves:
   - Team & Formation: Team information and the formation, for example 4213 and 442
   - Player & Attributes: Player positions and abilities, for example passing, defending and shooting
   - Tactical instructions: Passing style, pressing strategy and defensive depth

The football tactics model conforming to FootballMM will describe a team's formation, players, attributes and tactics.

2. **Simulation Metamodel (SimulationMM)**
   SimulationMM represents the simulation domain and defines how tactics affect the simulation rules:
   - Simulation and Agent: Simulation configuration and representation of players.
   - Pitch Zones: Areas such as defence, midfield and attack.
   - Simulation Rules: Rules for possession, passing, movement shooting and ball recovery.
   - Match Parameters: Number of simulations runs and random seed.

   For example, counterattack can favour a team with solid backline, low defensive line and fast attackers. The low press can decrease the risk of conceding goals but also gives less possession.

3. **Results Metamodel (ResultsMM)**
   ResultsMM represents the match results domain:
   - Match statistics: Possession, pass success, scoring chances and goals.
   - Results are structured so they can be mapped back to FootballMM.

   The match results model conforms to ResultsMM.

- *How many domains, MMs, and their relations.*
  This project involves three domains and three metamodels as described above. A T2M transformation creates the football tactics model (ftm) from a description of teams, formation, players, attributes and tactics. An M2M transformation maps the ftm, together with a set of base simulation rules, to a simulation model. An M2T transformation generates Python code from the simulation model. Python, as the external tool, executes the generated code and exports a CSV report. The CSV report is loaded into a match results model conforming to ResultsMM by a T2M, and a second M2M transformation maps the results model back to FootballMM.

  The artefacts are the ftm, simulation model, generated Python code, CSV report and match results model

- *Tool integration - external tools that produce or consume models. model compliance*
  The project will use:
  - Eclipse: Define metamodels and create models
  - QVTo: Implement the M2M transformations
  - Acceleo: Implement the M2T transformations
  - Python: Execute the generated simulation
  - CSV importer: Loads the CSV report into a match results model
  - VS Code: Inspect and debug the generated code

Model compliance will be checked by ensuring that each model conforms to its metamodel. Football models must contain valid positions and tactical instructions, while simulation models must contain valid rules and parameters. The generated Python code must be executable. The results are exported as a CSV report. Match results model must contain valid statistics.

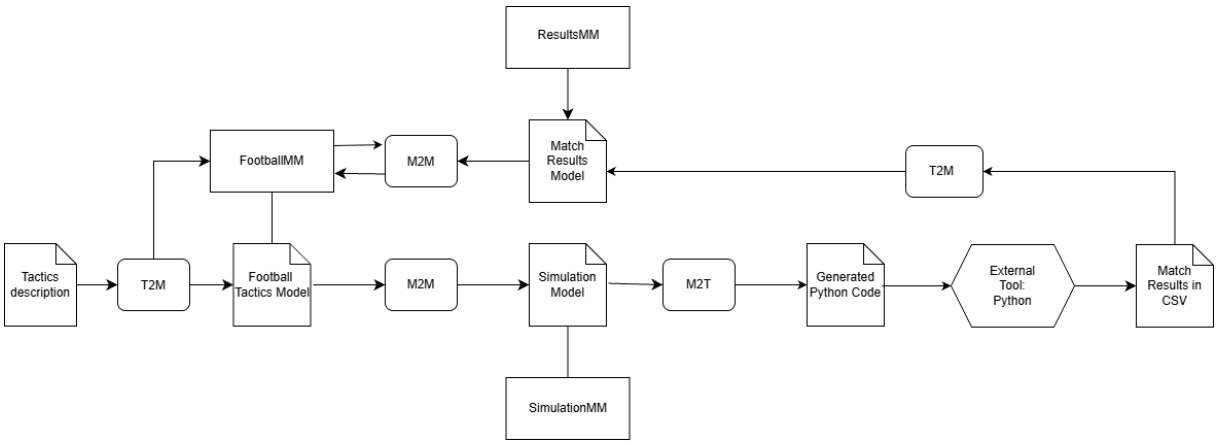


Figure shows the metamodels, models and transformations for the project

- *How will the model be created, updated?*

The football tactics model is created by the T2M transformation from a textual description of teams, formation, players, attributes and tactics, conforming to FootballMM. Existing tactics can also be used as input. The model can be updated manually whenever the user wants to test different tactics, or through the feedback M2M from the match results model. The simulation model is generated by the first M2M from the football tactics model and a set of base simulation rules, conforming to SimulationMM. The base rules are written by hand once. When the football model changes, the transformations can be rerun to regenerate the simulation model and the Python code. The match results model is created after each run by loading the CSV report, conforming to ResultsMM.

Transformations

- *What transformation will you develop*
  There are five different transformations:

1. **T2M:** Takes the description of teams, formation, players, attributes and tactics, as well as existing tactics, and creates a football tactics model conforming to FootballMM.
2. **M2M:** Takes the football tactics model and the base simulation rules as input. Teams are transformed into simulations, players into agents, formations into initial positions, and tactical instructions into simulation rules and parameters. For example, pressing affects ball recovery probability. The result is a simulation model.
3. **M2T:** Takes the simulation model and generates executable Python code that simulates match events and calculates statistics such as possession, chances and goals.
4. **T2M:** Takes the CSV report produced by Python and creates a match results model conforming to ResultsMM.
5. **M2M:** Takes the match results model and maps it back to FootballMM, so the results can inform changes to the tactics.

- *How will you combine transformations?*
  The transformations will be done in the following sequence:
  1. The user describes or updates the football tactics and the T2M creates the football tactics model.
  2. The first M2M maps the football tactics model and the base simulation rules to a simulation model.
  3. The M2T generates the Python code from the simulation model.
  4. Python executes the simulation and produces match results that are exported as a CSV report.
  5. The T2M loads the CSV report as a match results model.
  6. The feedback M2M maps the match results model back to FootballMM.
  7. The user can change tactics and repeat the process to compare results.

  Each transformation can execute independently if its input models are available. The complete process could eventually be automated so that all transformations run in sequence.